Project name: Automated Metadata Inference Platform for ISO 19139-Compliant OGC Open API Resources

Short project description, motivation, and expected outcome:
Research geospatial datasets, even when published, often lack metadata that renders them searchable using ISO standard geospatial data APIs (specifically ISO 19115/19139 and INSPIRE-aligned metadata profiles to maintain compatibility with OGC API–Features, Coverages, and Records). The DIWA DataLake is trying to automate metadata generation for research data storage to enable discoverability, usability, and interoperability of data. This project builds a web-based UI and API that automatically infers ISO 19139 metadata fields from whatever source metadata and data structure is already available in an OGC API resource, and forces explicit user input for any required field that cannot be inferred or is incomplete — guaranteeing every published record is schema-complete rather than partially populated.

Expected outcomes: Field-inference engine mapping available OGC API metadata/data structure to ISO 19139 fields; gap-detection logic flagging missing/incomplete required fields; web UI presenting inferred fields for review and forcing input on flagged gaps before submission; REST API for programmatic inference + submission; schema validation before publication; catalog/publish endpoint (OGC API – Records compliant) exposing completed records; search UI leveraging OGC API allowing users to identify data by area of interest, period of interest, and keyword search.

Platform/technical requirements: OGC API client libraries (Features, Coverages, Records), ISO 19139/19115 XML schema and validation tooling (e.g., pygeometa), backend web framework (Django), database for metadata record storage/versioning, frontend with form-validation/gap-flagging logic, deployment on VRE-hosted infrastructure

Expected completion date: 31 December 2026

Group size: 3–5 students

Required background and skills: Python (backend, XML/schema processing), web development (frontend + REST API, form validation), familiarity with OGC API (can be learned during project), XML/XSD and metadata standards (ISO 19115/19139, can be learned during project), database design

Goal: modify current data upload UI to force metadata generation. When a file is uploaded, metadata fields are read from the file directly, and a partially filled form is displayed to user. They can correct incorrectly inferred values, and must manually fill out key fields to complete upload of file. Once all fields have been filled out, another popup window forces confirmation of location, time, and distribution of values in file itself.

Tasks:

1) **Automatic metadata field inference from uploaded files**

Write a python application that autofills ISO 19139 + fields in data upload UI using information inferred directly from uploaded files. This should include inference of min, max, mean, and nodata identifiers

Background information:

- zipped file containing a variety of different filetypes
- DIWA Upload Portal https://fip-86-50-231-86.kaj.poutavm.fi/
- DigitalWaters-fi GitHub organization
- https://gdal.org/en/stable/
- https://exif.readthedocs.io/en/latest/usage.html

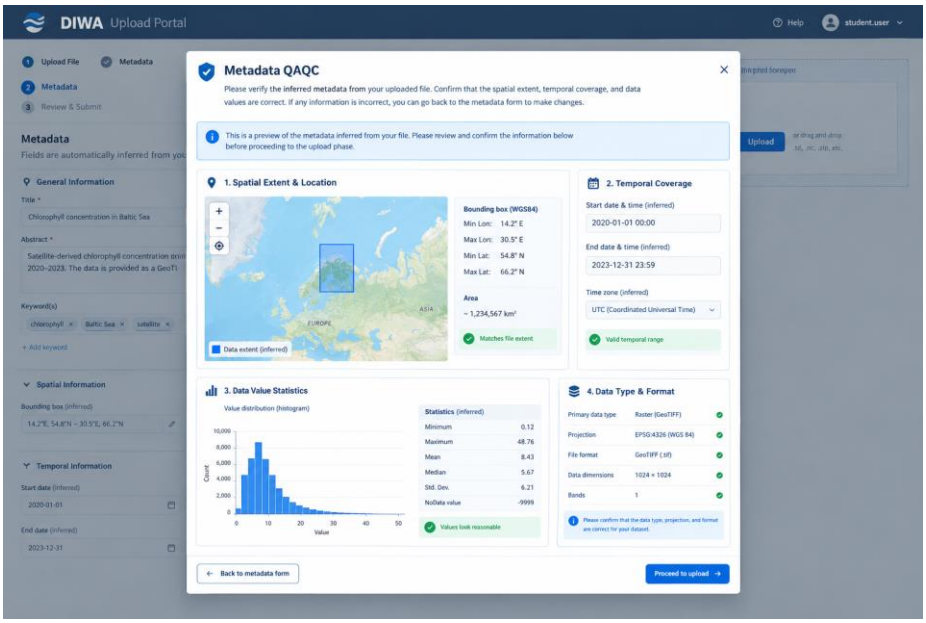
3) **Metadata form in the DIWA Upload Portal**

A modification of the existing upload workflow so that uploading a file triggers inference and opens a partially populated form. Inferred values are editable and visually distinguished from user-entered values. Fields flagged by gap detection block submission until completed. Controlled vocabularies are offered as selection lists rather than free text, including ISO topic categories and a keyword thesaurus such as GEMET. Validation runs both client-side and server-side.

4) Metadata QAQC screen

Build a graphical dashboard popup that shows metadata location on a world map, start and end date and time zone, and a histogram or box and whisker plot of values from uploaded dataset. Force users to validate that these fields are correct. To exit popup, they either go back to metadata form (and correct errors) or proceed to upload phase.

Example from ChatGPT:



The user has two exits only: return to the form to correct errors, or confirm and proceed to upload. The confirmation is stored with user and timestamp as part of the record history.

5. Schema validation before publication

Each completed record is serialised to ISO 19139 XML through pygeometa and validated against the XSD, with optional Schematron or INSPIRE validator checks.

6. Persistent identifier and data access URL

Each record receives a stable identifier and a resolvable access URL, written into the distribution information of the record. Candidate mechanisms will be assessed with the DIWA team, TBD. Stable URLs from Allas storage can be used as proxy.

7. REST API

Endpoints for:

- inference: submit a file, receive a draft record and gap report
- validation
- submission
- record retrieval

The API is documented with OpenAPI. Programmatic submission is subject to the same gap detection and validation as the web form and cannot bypass either.

8. Record storage and versioning

A PostgreSQL/PostGIS schema storing metadata records, their version history, provenance tags for inferred fields, and QAQC confirmations.

9. Catalogue endpoint

pygeoapi configured with an OGC API - Records provider exposing published records and supporting the `bbox`, `datetime` and `q` query parameters. Because OGC API - Records uses a JSON/GeoJSON record model, the ISO 19139 XML is offered as an alternate downloadable representation of each record.

10. Map-based data discovery UI

A search interface built on the Records endpoint:

- users draw an area of interest, upload a shapefile or bounding box defining geographic region, set a time window, and enter keywords
- results appear as a list with footprints drawn on the map
- each result opens a record detail view with a link to the data through its persistent URL
- User can select files, UI generates bash download script
- Option to click to download?